

6

Manipulating strings

This chapter demonstrates various ways to manipulate text strings in PHP scripts.

Comparing characters

Searching text

Extracting substrings

Changing case

Formatting strings

Making dates

Encoding entities

Summary

Comparing characters



PHP provides a number of built-in functions to perform a comparison of two text strings. The simple **strcmp()** function accepts two string arguments for comparison and will return one of three integer values depending on the result. When the strings exactly match, **strcmp()** will return zero. When the first string argument is greater than the second, **strcmp()** will return 1, but when the first string argument is less than the second, **strcmp()** will return -1.

The **strncmp()** function works in a similar way, but accepts a third integer argument to specify the number of characters from the start of the strings to be compared.

The **strcmp()** and **strncmp()** functions attach different importance to uppercase and lowercase characters when comparing strings. When you prefer to ignore case, you can use the alternative companion functions **strcasecmp()** and **strncasecmp()** instead.



Strictly speaking, the **strcmp()** returned values will be a positive integer, zero, or a negative integer – but are typically values of 1, 0, or -1.

To appreciate the results from each of the four comparison functions it is useful to understand how the comparisons are actually made. Each character has a standard numerical “ASCII” value where lowercase a-z is in the range 97-122 and uppercase A-Z is in the range 65-90. Comparing the first value in each range we see that “a” (97) is greater than “A” (65). Supplying these as arguments, the **strcmp()** function would therefore return 1, as the first argument is greater than the second. Conversely, if you specify “A” (65) as the first argument and “a” (97) as the second argument then the **strcmp()** function would return -1, as the first argument is less than the second. With string comparisons, each character is compared in sequence to ensure that strings containing the same characters in a different order are not seen to be

incorrectly equal. For example, comparing “ABC” and “BAC” the first character “A” (65) is less than first character “B” (66) so the **strcmp()** function would return -1, as the first argument is less than the second.



ASCII is an abbreviation for the American Standard Code for Information Exchange that specifies the universal standard character coding scheme.

The ASCII value of any character can be found by specifying the character as an argument to the PHP **ord()** function, and the length of a string can be found by specifying it as the argument to the PHP **strlen()** function.

1 Begin a PHP script with statements to initialize four variables with string values

```
<?php
```

```
$str1 = 'PHP in easy steps' ;
```

```
$str2 = 'PHP in easy steps' ;
```

```
$str3 = 'PHP In Easy Steps' ;
```

```
$str4 = 'php in easy steps' ;
```



characters.php

2 Next, add statements to compare the strings

```
echo "$str1' versus '$str2' : ".strcmp($str1, $str2). '<br>' ;
```

```
echo "$str1' versus '$str3' : ".strcmp($str1, $str3). '<br>' ;
```

```
echo "$str1' versus '$str4' : ".strcmp($str1, $str4). '<hr>' ;
```

3 Now, add a statement for a case-insensitive comparison

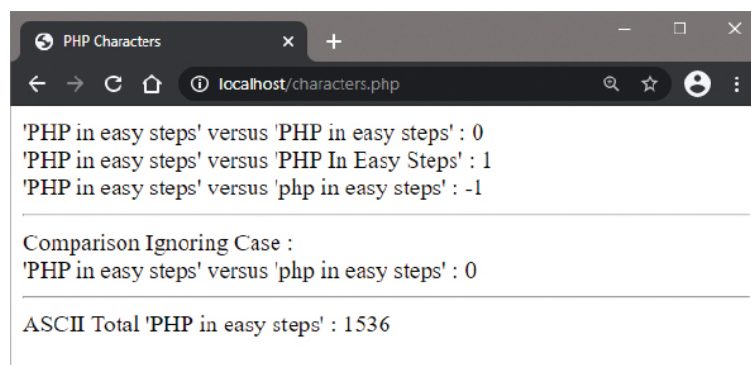
```
echo 'Comparison Ignoring Case :<br>' ;
```

```
echo "$str1' versus '$str4' : ".strcasecmp($str1, $str4) ;
```

4 Finally, add a loop to count and display the total ASCII code value of the first string

```
$total = 0 ;
for ( $i = 0 ; $i < strlen( $str1 ) ; $i++ )
{
    $total += ord( $str1[ $i ] ) ;
}
echo "<hr>ASCII Total '$str1' : $total" ;
?>
```

5 Save the document in your web server's **/htdocs** directory as **characters.php** then open the page via HTTP to see the string comparisons



Character order is relevant as the ASCII totals for two differently ordered strings could be equal, but the strings would not match.

Searching text



PHP provides a number of built-in functions to search a string to find a specified substring or single character. The simple **strpos()** function accepts two arguments to specify the string in which to search, and the substring to find. It will search in sequence from the start of the string to find a match. When a match is found, the search halts and the **strpos()** function returns an integer, which is the character index number position within the string at which the first occurrence of a match begins. When no match is found, the **strpos()** function returns **FALSE**.

The **strrpos()** function works in a similar way but returns the index number position at which the first occurrence of a match begins when searching in reverse order, from the end of the string.

Other PHP search functions can return a part of the searched string, rather than an index number indicating a match position.

The **strstr()** function accepts two arguments to specify the string in which to search, and the substring to find. It will search from the start of the string to find a match. When a match is found, the search halts and the **strstr()** function returns the remainder of the searched string from where the first occurrence of a match begins. When no match is found, the **strstr()** function returns **NULL**.



Notice that there is no **strrstrstr()** (companion function to **strstr()**) to search in reverse order.

Similarly, where character case is unimportant, you can use the companion **striistr()** function to make a case-insensitive search. This will return the remainder of the searched string from where the first occurrence of a case-insensitive match begins.

The **strchr()** function works in a similar way but accepts two arguments to specify the string in which to search, and a single character to find. The companion **strrchr()** function begins searching in reverse order, from the end of the string. Both return the remainder of the searched string from where the first occurrence of a match begins.

You can also seek a match by specifying the string to search and the substring to find as arguments to **str_contains()**, **str_starts_with()** and **str_ends_with()** functions. These each return a Boolean **TRUE** or **FALSE** value according to the search result. The return can assign an appropriate string to describe the result.

1 Begin a PHP script with statements to initialize a variable with a string value, then report its length `<?php`

```
$str = 'Most Users usually find PHP useful.' ;
echo "$str"<br>String Length : ".strlen( $str ) ;
```



search.php

2 Next, add statements to search the string and report the position of a given substring

```
echo "<br>First 'us' found at : ".strpos( $str , 'us' ) ;
echo "<br>Final 'us' found at : ".strrpos( $str , 'us' ) ;
```

3 Now, add statements to search the string and return the remainder of the string following a match

```
echo "<hr>Substring from first 'us' : ".strstr( $str , 'us' ) ;
echo "<br>Characters from final 'u' : ".strrchr( $str , 'u' ) ;
```

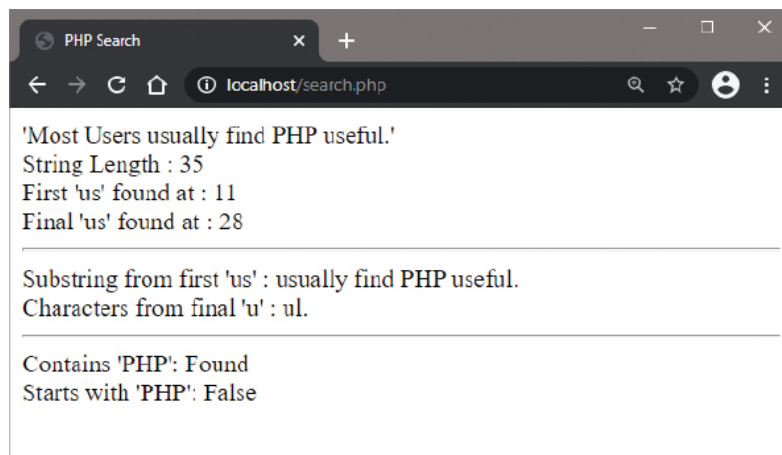
4 Finally, add statements to search the string, and assign appropriate strings to describe the search result

```
$result = str_contains( $str , 'PHP' ) ? 'Found' : 'Absent' ;
echo "<hr>Contains 'PHP': ".$result ;

$result = str_starts_with( $str , 'PHP' ) ? 'True' : 'False' ;
echo "<br>Starts with 'PHP': ".$result ;

?>
```

5 Save the document in your web server's **/htdocs** directory as **search.php** then open the page via HTTP to see the search results



The **str_contains()**, **str_starts_with()** and **str_ends_with()** functions are great new features introduced in PHP 8.



There are two occurrences of **'us'** in this string. The initial **'Us'** is not matched by the case-sensitive searches.

Extracting substrings



A substring can be extracted from a string by stating the string, and index position of a substring within the string, as arguments to the **substr()** function. Optionally, a third argument can specify the substring length. You may replace a substring using the **substr_replace()** function to specify string, substring, index position and length arguments. The total number of occurrences of a substring within a string can be found using the **substr_count()** function:

- 1 Begin a PHP script with statements to initialize a variable with a string value and count substrings

```
<?php
```

```
$str = 'SQL in easy steps features SQL queries' ;
```

```
echo "'$str'<br>'SQL' Count: ".substr_count( $str , 'SQL' ) ;
```



substring.php

- 2 Next, add statements to extract two substrings

```
echo '<hr>Index 27 : '.substr( $str , 27 ) ;
```

```
echo '<hr>Index 4 Length 13 : '.substr( $str , 4 , 13 ) ;
```

- 3 Now, add a statement to initialize a variable with a string

```
$sub = 'PHP & MySQL' ;
```

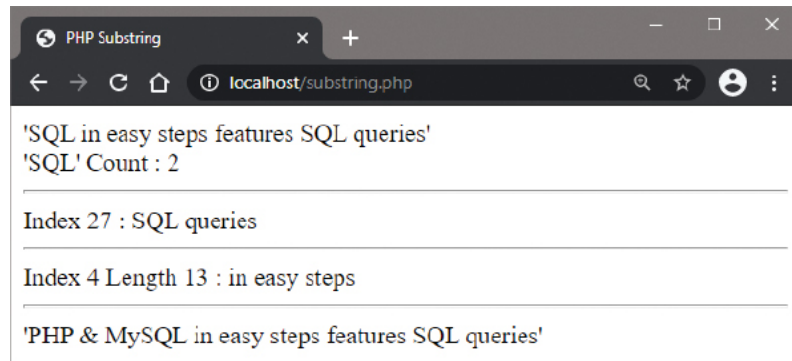
- 4 Finally, add statements to replace a substring with a new string value and to display the modified string

```
$str = substr_replace( $str , $sub , 0 , 3 ) ;
```

```
echo "<hr>'$str'" ;
```

?>

5 Save the document in your web browser's **/htdocs** directory as **substring.php** then open the page via HTTP to see the extracted substrings and modified string



The length argument in **substr_replace()** specifies the length of the substring to be replaced. In this example, it is **3** – to replace the three characters **'SQL'** with a longer string.

Changing case



PHP provides functions to easily modify the character case of a specified string argument. The **strtolower()** function changes all characters to lowercase, and **strtoupper()** changes them all to uppercase. Additionally, the **ucfirst()** function makes only the first string character uppercase, whereas the **ucword()** function makes the first character of every word in the string uppercase:

1 Begin a PHP script with statements to initialize a variable with a string and display its value

```
<?php
```

```
$str = "C++ Programming in easy steps" ;
```

```
echo "Original String : '$str' <hr>" ;
```



case.php

2 Next, add statements to display a lowercase version

```
$ver = strtolower( $str ) ;
```

```
echo "Lowercase String : '$ver' <br>" ;
```

3 Now, add statements to display an uppercase version

```
$ver = strtoupper( $str ) ;
```

```
echo "Uppercase String : '$ver' <hr>" ;
```

4 Finally, add statements to display a capitalized string version and a capitalized words version

```
$ver = ucfirst( strtolower( $str ) ) ;
```

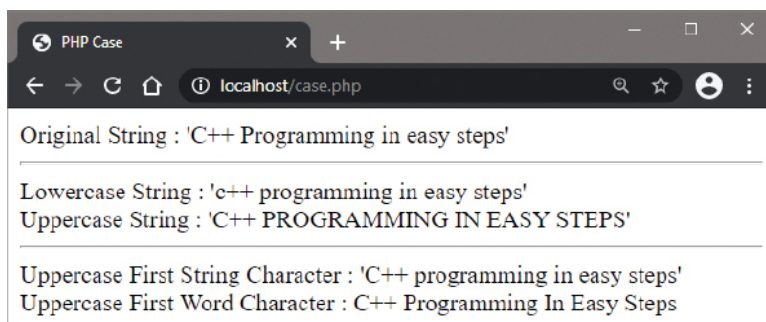
```
echo "Uppercase First String Character : '$ver' <br>" ;
```

```
echo 'Uppercase First Word Character : 'ucwords( $ver ) ;
```

```
?>
```

5

Save the document in your web server's **/htdocs** directory as **case.php** then open the page via HTTP to see the changed case versions of the original string



The **ucfirst()** function only changes the first character of a string. Other uppercase characters will remain unless first changed to lowercase with the **strtolower()** function, as seen here.

Formatting strings



PHP provides a number of built-in functions to conveniently format strings by manipulating their characters. The simple **strrev()** function accepts a single string argument and returns that string in reversed character order, so it reads back to front.

The **strrpt()** function accepts two arguments to specify a string, and an integer of how many total times that string should be repeated in the string it will return.

The **strpad()** function accepts three arguments to specify a string, a total length of the character string it will return, and a character or substring pattern with which to fill the difference between the original string and the specified returned string length. The padding will, by default, be added to the end of the original string but can be added to its start by adding a fourth argument using a special **STR_PAD_LEFT** flag.

The **trim()** function can accept a single string argument and will return a string in which all white-space characters (spaces, line returns, tabs) have been removed from the start and end of the original string. Optionally, a second argument can specify a character or pattern to be removed from the start and end of the original string, in place of the white-space default.

Similarly, where you only want to remove characters from the start of the original string, you can use the companion **ltrim()** function, or where you only want to remove characters from the end of the original string, you can use the companion **rtrim()** function.



The PHP **implode()** and **explode()** functions can be used to convert arrays to strings, and strings to arrays – see here for more details.

A string can be split into individual parts using the **strtok()** function to specify the string and a delimiter token around which the parts should be split. For example, if you want to split a string into individual words, you can specify the space character as the delimiter token. The first part of the string, up to the delimiter token, will be returned by the function. Interestingly, the **strtok()** function remembers its position in the string, so subsequent calls need only specify the delimiter token to return the next part of the string, until no further delimiters are found. Splitting a string in this way can best be achieved using a loop to make subsequent calls to the **strtok()** function, until it reaches the end of the string.

1 Begin a PHP script with statements to initialize a variable with a string and display its value

```
<?php
```

```
$str = '| PHP String Fun |' ; echo "Original String : $str" ;
```



format.php

2 Next, add statements to display reversed, repeated and trimmed versions of the original string

```
echo '<hr>Reversed String : '.strrev( $str ) ;
```

```
echo '<hr>Repeated String : '.str_repeat( $str , 3 ) ;
```

```
echo '<hr>Trimmed String : '.trim( $str, '|' ) ;
```

3 Now, add a statement to display a 30-character version of the string, padded with asterisks at the beginning

```
$pad = str_pad( $str , 30 , '*' , STR_PAD_LEFT ) ;
```

```
echo "<hr>Padded String : $pad" ;
```

4 Finally, add statements to split the string around spaces and display each individual part separated by an ... ellipsis

```
echo '<hr>Split String : ' ;
```

```
$token = strtok( $str , ' ' ) ;
```

```
while ( $token )
```

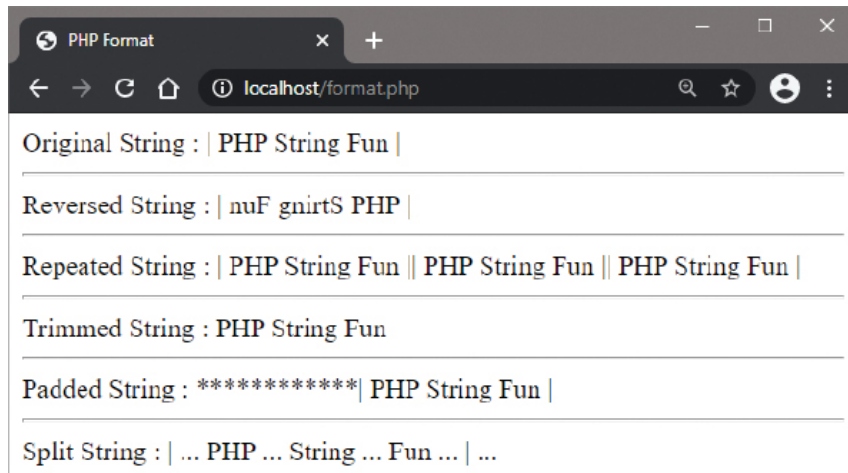
```
{
```

```
    echo "$token ... " ; $token = strtok( ' ' ) ;
```

```
}
```

?>

5 Save the document in your web server's **/htdocs** directory as **format.php** then open the page via HTTP to see the string formatted in several ways



You must specify the delimiter on each subsequent call to **strtok()** to have it continue forward to the next part of the string.

Making dates



The PHP **date()** function will return a date string when called, but requires a string argument to specify a format for the date string. This format string can comprise a series of letter options for date and time. For example, **date('n/j/y')** might return “4/22/16”. Popular options are listed below along with a brief description:

Option	Description	Example
Y	Year as four digits	2023
y	Year as two digits	14
n	Month as one or two digits	7
m	Month as two digits	07
F	Month full name	February
M	Month name as three letters	Feb
j	Day as one or two digits	4
d	Day as two digits	04
l	Weekday full name	Monday
D	Weekday as three letters	Mon
S	Ordinal suffix as two letters	th
H	Hour in 24-hour format	14
i	Minutes	45
s	Seconds	30

a

Ante or post meridiem

am

The weekday full name option is a lowercase “L”.

By default, the timezone is set to the Universal Time Clock (UTC) but you can set it to your own timezone by specifying it as an argument to the **date_default_timezone_set()** function, and see the currently-set timezone with **date_default_timezone_get()**.

You may also create your own timestamp using the intelligent **strtotime()** function. This can accept a date string argument and return a timestamp that can be supplied as an optional second argument to the **date()** function. The argument specified to **strtotime()** can be a regular text date, such as “July 4, 2023”. Additionally, you can specify an argument such as “yesterday”, “+3months” or “next Wednesday” and the **strtotime()** function will try to figure it out and return an appropriate timestamp.



You can discover more **date()** function options in the PHP manual online at php.net/manual



Begin a PHP script with statements to display the current date, day, time and current default timezone

```
<?php
```

```
echo 'Date : '.date( 'jS F Y' ).'<br>' ;
```

```
echo 'Day : '.date( 'l' ).'<br>' ;
```

```
echo 'Time : '.date( 'h:i:s a' ).'<br>' ;
```

```
echo 'Timezone : '.date_default_timezone_get().'<br>' ;
```



date.php

2 Next, add statements to change the default timezone and display a confirmation, then show the current time there

```
date_default_timezone_set( 'America/New_York' );
echo 'Timezone now : '.date_default_timezone_get();
echo '<br>Time now : '.date( 'h:i:s a' ).'<br>' ;
```

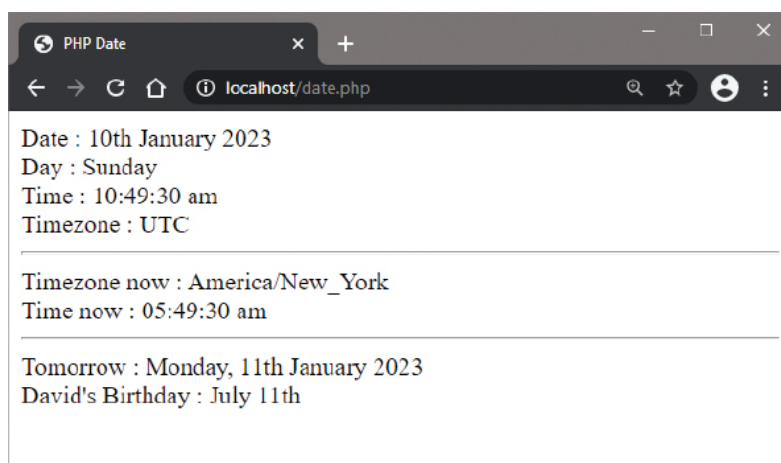
3 Now, add statements to create a timestamp for the next day, then display its day name and date

```
$d = strtotime( 'tomorrow' );
echo 'Tomorrow : '.date( 'l, jS F Y' , $d ).'<br>' ;
```

4 Finally, add statements to create a timestamp for a memorable date, then display its month and day number

```
$d = strtotime( 'July 11, 1994' );
echo 'David\'s Birthday : '.date( 'F jS' , $d );
?>
```

5 Save the document in your web server's **/htdocs** directory as **date.php** then open the page via HTTP to see the date strings formatted by the options



Notice that you can include separator characters such as : and / in the format string.



Some countries prefer date order of M/D/Y while others prefer D/M/Y so you can choose your preferred date format options.

Encoding entities



There are two PHP functions available to encode HTML code, so that all characters of special significance or unusual characters are replaced by “entity” equivalents. The **htmlentities()** function encodes all characters below, which each have an entity equivalent:

" & < > ¡ ¢ £ ¤ ¥ ¦ § ¨ © ª « ¬ ® ¯ ° ± ² ³ ´ µ ¶ · ¸ ¹ º » ¼ ½ ¾ ¿ À Á Â Ã Ä Å Æ Ç È É Ê Ë Ì Í Î Ï Ð Ñ Ò Ó Ô Õ Ö × Ø Ù Ú Û Ü Ý Þ ß à á â ã ä å æ ç è é ê ë ì í î ï ð ñ ò ó ô õ ö ÷ ø ù ú û ü ý þ ÿ Æ œ Š š Ÿ ƒ ˆ ˜ A B Γ Δ E Z H Θ I K Λ M N Ξ O Π P Σ T Υ Φ X Ψ Ω α β γ δ ε ζ η θ ι κ λ μ ν ξ ο π ρ σ τ υ φ χ ψ ω ϑ Υ ϖ — ‘ ’ , “ ” „ † ‡ • … ‰ ‹ › ⁄ € § ¨ ™ ® ← ↑ → ↓ ↔ ↵ ⇐ ⇑ ⇓ ⇔ ∇ ∅ ∇ ∈ ∉ ∋ ∏ ∑ − ∗ √ α α ∠ ∨ ∩ ∪ ∴ ∴ ∼ ∼ ∼ ≈ ≠ ≡ ≤ ≥ ⊂ ⊃ ⊄ ⊆ ⊇ ⊕ ⊗ ⊥ ∙ [] [] () ∅ ♠ ♣ ♥ ♦

Unless you are concerned about non-English language characters, it is generally preferable to use the alternative **htmlspecialchars()** function to encode only characters of special HTML significance:

- & ampersand – becomes **&**;
- “ double quote – becomes **"**;
- ‘ single quote – becomes **'**;
- < less than – becomes **<**;
- > greater than – becomes **>**;



Call the PHP function **get_html_translation_table()** with the constant argument of **HTML_ENTITIES** or **HTML_SPECIALCHARS** to return an array of all represented characters.

One good reason to encode HTML snippets with PHP is to prevent malicious JavaScript code being executed unintentionally by the web browser. Encoding HTML tags into entities ensures that the HTML **<script>** tag will not be recognized, so its JavaScript code contents will not be executed:

- 1 Create a PHP script with statements to initialize a variable with an HTML snippet and display its value

```
<?php
```

```
$html = '<script>>window.location="index.html"</script>' ;
```

```
// Statement to be inserted here.
```

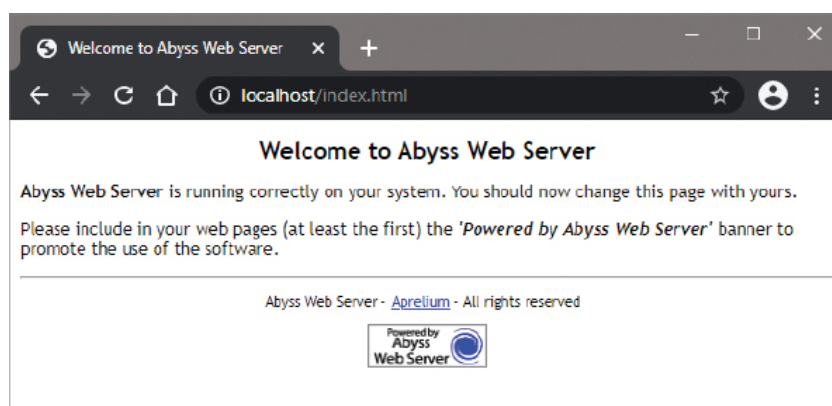
```
echo $html ;
```

```
?>
```



encode.php

- 2 Save the document in your web server's **/htdocs** directory as **encode.php** then open the page via HTTP to hopefully see the HTML code snippet

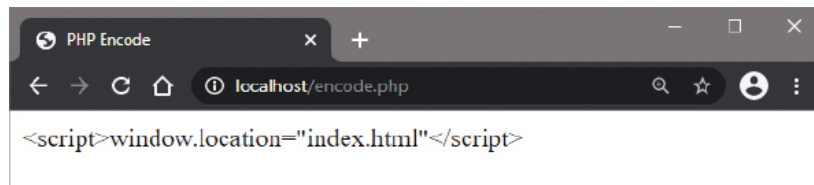


The browser has interpreted the code and relocated to the server's index page because it recognized the HTML **<script>** tag.

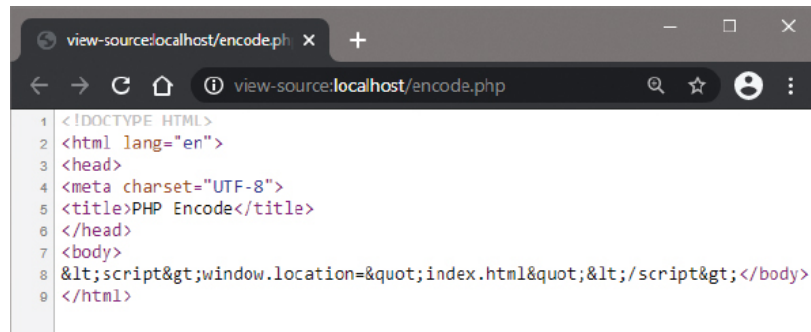
- 3 Return to the PHP script and insert a statement to encode special HTML characters

```
$html = htmlspecialchars( $html ) ;
```

- 4 Save the edited document and reopen the page via HTTP again to see the HTML code snippet



5 Use the **View Source** facility in your web browser to see the special character entity equivalents encoded by PHP



The PHP function **html_entity_decode()** can be used to unencode all entities back to their original character state.

Summary

- The PHP built-in string comparison functions **strcmp()**, **strncmp()**, **strcasecmp()** and **strncasecmp()** each typically return a result of 1 (greater), 0 (equal) or -1 (less).
- String comparisons are based on the numerical ASCII value of each character, compared in sequence to account for order.
- The ASCII value of any character can be found using **ord()**.
- The length of any string can be found using **strlen()**.
- The numerical index position of a substring within a searched string can be found using **strpos()** and **strrpos()** functions.
- The search functions **strstr()**, **stristr()**, **strchr()** and **strrchr()** each return the remainder of a searched string on success.
- The substring functions **substr()**, **substr_replace()** and **substr_count()** can extract, replace or count occurrences.
- The characters in a string can be reversed using **strrev()**, or the string can be repeated using **strrpt()**.
- Padding characters can be added to a string using **strpad()** and characters can be removed using **trim()**, **ltrim()** or **rtrim()**.
- A string can be split into individual parts by specifying a delimiter to the **strtok()** function.
- The **date()** function requires a string argument that specifies options to determine the format of its returned date string.
- By default, the PHP date timezone is set to UTC.
- The **date_default_timezone_set()** function can be used to specify an alternative to the UTC default timezone.
- The **strtotime()** function will attempt to return an appropriate timestamp for whatever date or time argument it receives.
- The **htmlentities()** function will encode all string characters for which there is an equivalent entity.

- The **htmlspecialchars()** function will encode only characters of special HTML significance to avoid recognition by a browser.